

App Development Project Report

1. Student Details

Name: Panchavarnam Baskaran Nadar

Roll Number: 24f2008200

Email: 24f2008200@ds.study.iitm.ac.in

About Me: I am passionate about building practical, scalable web applications with clean architecture and meaningful user experiences. Through this project, I explored modular Flask design, reusable templates, and integrated data handling to develop a realistic end-to-end system.

2. Project Details

Project Title: Doctor–Patient Appointment Management System

Problem Statement: To design and implement a hospital appointment management system that allows administrators, doctors, and patients to coordinate efficiently. The goal is to enable seamless scheduling, transparent slot availability, and real-time interaction among all stakeholders.

Approach: The application is built using Flask as the backend framework with modular blueprints for Admin, Doctor, and Patient roles. Data is managed through SQLAlchemy ORM integrated with an SQLite backend. The user interface is designed using Jinja2 templates and Bootstrap for responsiveness and consistency. Redis and Celery are integrated to handle caching and batch jobs for reminders and notifications. The focus of the project was on ensuring data integrity, role-based functionality, and system extensibility for future scaling.

3. AI / LLM Declaration

I used ChatGPT (GPT-5) to assist in refining route logic, organizing modular structures, and drafting this project documentation. The extent of AI/LLM usage is around 20–25%, limited to refactoring suggestions, code readability improvements, and formatting of the report. All database modeling, debugging, and testing were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend framework
Flask-Login	User authentication and session management

Flask-SQLAlchemy	ORM for database operations
Jinja2	Dynamic templating engine
Bootstrap 5	Frontend styling and responsive design
Redis + Celery	Caching and asynchronous background task handling
SQLite	Lightweight local relational database
Chart.js / Iframes	Embedding analytics and visual data components

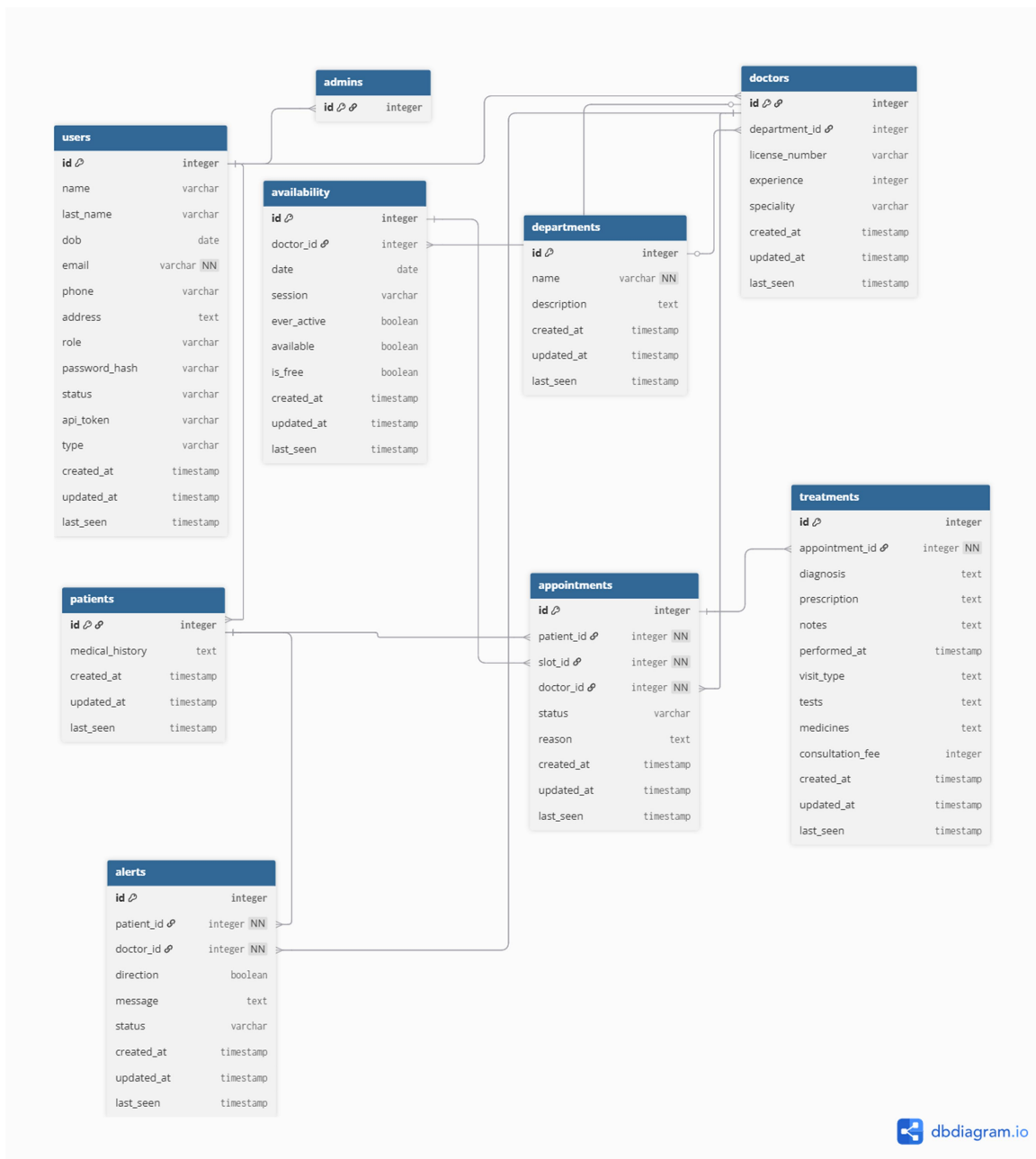
5. Database Schema / ER Diagram

Tables:

- User — Common user details (id, name, email, role, password)
- Doctor — Doctor profile linked to Department and User
- Patient — Patient profile linked to User
- Department — Medical specialization categories
- Slot — Doctor's available time slots
- Appointment — Links Patient and Doctor via Slot with status and billing details
- Alert — Notifications and communication records

Relationships:

- One-to-Many → Department → Doctor
- One-to-Many → Doctor → Slot
- One-to-One → Slot → Appointment
- One-to-Many → Patient → Appointment
- One-to-One → User → Doctor / Patient



(ER Diagram created using dbdiagram.io or draw.io)

6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user and start session
/admin/dashboard	GET	Admin control panel showing doctors, patients,

		and appointments
/doctor/dashboard	GET	Doctor dashboard for slots and patient interactions
/patient/dashboard	GET	Patient dashboard to view or book appointments
/appointment/book	POST	Book a slot for a patient with a doctor
/appointment/cancel	POST	Cancel an existing appointment
/alerts	GET	Fetch alerts for doctors and patients
/analytics	GET	Visual analytics and summary reports

7. Architecture and Features

Architecture Overview:

- app.py – Main Flask entry point
- /models – SQLAlchemy model definitions
- /routes – Modular Flask Blueprints (Admin, Doctor, Patient)
- /templates – Jinja2 HTML templates
- /static – Bootstrap CSS, JS, and icons
- /database – SQLite database file
- /tasks – Celery and Redis job handlers for background jobs

Core Features:

- Role-based access control (Admin, Doctor, Patient)
- Doctor schedule and slot management
- Appointment booking and cancellation with automatic slot release
- Patient and Doctor dashboards with summaries
- Alerts and notifications for status changes
- Real-time data consistency across roles
- Secure login and session handling with Flask-Login

Additional Features and Highlights:

1. Smart Appointment Algorithm — Ensures that both the doctor and the patient are free before confirming a booking, avoiding overlapping appointments.
2. Reusable Template Macros — Macros were used to recreate dynamic sections of templates for different use cases, reducing redundancy.
3. Modular and Component-Based UI — Pages were generated using modular structures,

enhancing maintainability and reusability.

4. Filter Tables — Implemented client-side filters to extract relevant rows from large datasets, improving data accessibility.

5. Iframes and API Integrations — Demonstrated embedding of APIs and Iframes to showcase how the system can scale using web services and micro-components.

8. Insights and Reflection

This project serves as a proof-of-concept demonstration, focusing on exploiting multiple technologies to meet customer experience and scalability goals. By no means is it the most optimal solution — the final production-grade system should be developed after a comprehensive user requirement analysis, leveraging the design insights and technologies explored here.

The project helped me deeply understand modular design, multi-role interaction in web apps, and how technology choices influence scalability and user experience.

9. Video Presentation

Drive Link:

<https://drive.google.com/file/d/your-video-link-here>

(Ensure link is accessible to 'Anyone with the link – View only')

10. Submission Instructions

This report is included as a .docx or .pdf in the submission ZIP file along with the complete project codebase. The same report (in PDF format) should be uploaded to Google Drive and the link pasted in the Viva Portal during submission.